Multimodal AI-Based Healthcare Diagnostic & Clinical Decision Support System (CDSS)

Major Project – Progress / Technical Report B.Tech Computer Science and Engineering (Cybersecurity), IILM University, Greater Noida Team members: *[add names and roll numbers]* | Guide: *[add name]*

> **Reading note.** Items marked **[TO FILL]** need your team's own evidence (screenshots, GPU model, timings). Nothing in this report is invented: every number comes from your training/evaluation output, and anything not yet built is clearly labelled as planned.

---

1. System Architecture and Design

1.1 Project Objective

The project aims to build a decision-support system that analyses chest X-ray images (and, in later phases, the paired radiology report text) to predict multiple radiological findings. The system is positioned as a **clinician-facing decision-support prototype, not an autonomous diagnostic device**.

1.2 Proposed (Target) Architecture

The target architecture follows a six-layer design:

Layer	Purpose	Components
1. Data Ingestion	Collect raw inputs	Chest X-ray images, radiology reports, optional structured clinical data
2. Preprocessing	Make inputs model-ready	Image resize/normalize/augment; text cleaning and tokenization; tabular imputation/scaling
3. Modality Encoders	Convert each input to a vector	Vision: DenseNet121 (1024-d); Text: Bio_ClinicalBERT (768-d); Tabular: MLP (optional)

Layer	Purpose	Components
4. Multimodal Fusion	Combine modality vectors	Concatenation (1024 + 768 = 1792-d) followed by an MLP head; cross-attention as a stretch goal
5. Clinical Decision Layer	Produce outputs	Multi-label finding classifier, confidence/risk estimator, explainability (Grad-CAM + attention)
6. Application Layer	Deliver to the user	FastAPI backend, clinician dashboard, audit log and feedback loop

1.3 Module Decomposition

Module	Responsibility	Status
M1 – Data loading and label construction	Merge image and report CSVs; build 14-label multi-hot targets	**Completed**
M2 – Patient-level splitting	Train/val/test split by patient (study) ID	**Completed**
M3 – Image preprocessing and augmentation	Resize, crop, flip, jitter, ImageNet normalization	**Completed**
M4 – Vision encoder/classifier	DenseNet121 fine-tuned for 14 findings	**Completed (baseline)**
M5 – Training and evaluation engine	Weighted BCE loss, Adam, per-label AUROC, best-checkpoint saving	**Completed**
M6 – Text encoder	Bio_ClinicalBERT report embeddings	Planned
M7 – Fusion model	Concatenation fusion head	Planned
M8 – Explainability	Grad-CAM, attention visualization	Planned
M9 – Calibration / confidence	Reliability diagram, temperature scaling	Planned
M10 – Backend API	FastAPI `/predict` endpoint	Planned

Module	Responsibility	Status
M11 – Website / front end	Clinician dashboard for upload and results	Planned

1.4 System Workflow

1. A clinician uploads a chest X-ray (and, in the full system, the clinical report text).
2. The backend preprocesses the image (resize to 224×224, ImageNet normalization) and tokenizes the text.
3. The vision encoder (and text encoder) produce feature vectors; the fusion head combines them.
4. A sigmoid layer converts logits to one probability per finding.
5. The explainability module produces a Grad-CAM heatmap for the selected finding.
6. The dashboard displays findings, confidence values and the heatmap.

1.5 Data Flow

`Raw Kaggle data → merge on uid → label construction → patient-level split → augmentation/tokenization → DataLoader batches → model training → evaluation (AUROC) → best checkpoint → packaged model → inference API → dashboard`

1.6 Technology Stack

Area	Technology
Language	Python 3
Deep learning	PyTorch, torchvision
Data handling	pandas, NumPy, Pillow (PIL)
Evaluation	scikit-learn (`roc_auc_score` , `train_test_split`)
Text (planned)	HuggingFace Transformers – Bio_ClinicalBERT
Explainability (planned)	pytorch-grad-cam
Serving (planned)	FastAPI, Uvicorn
Front end (planned)	Streamlit or React

Area	Technology
Training environment	Kaggle notebook (GPU when available); evaluation re-run in Google Colab

1.7 Key Design Decisions

Decision	Rationale
Transfer learning with ImageNet-pretrained DenseNet121	Only a few thousand medical images are available; pretrained filters (edges, textures) transfer well, and DenseNet121 is the CheXNet backbone
Multi-label formulation with `BCEWithLogitsLoss`	One X-ray can show several findings at once; each finding is an independent yes/no decision
Patient-level split (by `uid`)	A patient has multiple images; random image-level splitting leaks patient information and inflates scores
`pos_weight` (capped at 20)	Findings are rare, so positives are up-weighted to counter class imbalance without exploding the loss for very rare labels
AUROC as primary metric	Threshold-independent and robust to class imbalance; accuracy would be misleading
Unimodal baseline first	Needed to later prove, with numbers, that multimodal fusion adds value

1.5 Interfaces Between Modules

Interface	From → To	Format
I1	Data loader → Model	Tensor batch `[32, 3, 224, 224]` and label tensor `[32, 14]`
I2	Vision encoder → Fusion head (planned)	1024-d feature vector per image

Interface	From → To	Format
I3	Text encoder → Fusion head (planned)	768-d [CLS] vector per report
I4	Fusion head → API (planned)	14 logits → sigmoid probabilities
I5	API → Dashboard (planned)	JSON: `{finding: probability}` plus heatmap image

---

2. Methodology and Implementation

2.1 Algorithms and Models

Convolutional Neural Network (DenseNet121). Convolutional layers slide small learnable filters across the image, so the model learns spatial features (edges → textures → shapes → anatomical patterns) with few parameters. DenseNet121 connects every layer to all previous layers within a dense block, improving feature reuse. The ImageNet-trained weights (`IMAGENET1K_V1`) are loaded, and the 1000-class classifier is replaced with `nn.Linear(1024, 14)` .

Fine-tuning strategy. In the implemented baseline, all layers are trainable (no freezing) with a small learning rate (1e-4), which lets pretrained filters adapt gently to X-ray statistics.

Loss function. `BCEWithLogitsLoss(pos_weight=...)` , where for each label `pos_weight = min(negatives / max(positives,1), 20)` .

Optimizer. Adam, learning rate 1e-4.

Evaluation metric. Per-label ROC-AUC and macro mean AUROC (labels lacking both classes in a split are skipped).

2.2 Label Construction

The dataset's `reports` file holds a semicolon-separated `Problems` field per study. The implementation:

1. Merges `indiana_projections.csv` (image filenames) with `indiana_reports.csv` (Problems text) on `uid` .
2. Fills missing `Problems` with `normal` .

3. Counts all tags and selects the **14 most frequent non-normal tags** as label set.
4. Builds a multi-hot label matrix once in the Dataset constructor, using exact (case-insensitive) tag matching.

Selected labels: Lung, Opacity, Cardiomegaly, Calcinosis, Pulmonary Atelectasis, Calcified Granuloma, Thoracic Vertebrae, Cicatrix, Spine, Markings, Pleural Effusion, Aorta, Diaphragm, Density.

2.3 Preprocessing and Augmentation

Stage	Training	Validation / Test
Resize	256×256	224×224
Crop	Random crop 224×224	–
Flip	Random horizontal flip (p = 0.5)	–
Colour	Brightness and contrast jitter (0.1)	–
Tensor + normalize	ToTensor; ImageNet mean/std	ToTensor; ImageNet mean/std

Only horizontal flips are used because vertically flipped chest X-rays are not anatomically plausible.

2.4 Frameworks, Tools and Languages

- **Language:** Python.
- **Frameworks:** PyTorch and torchvision (model, transforms, DataLoader); scikit-learn (metrics, splitting).
- **Development tools:** Kaggle notebooks, Google Colab, GitHub for version control.

2.5 Database Design

The prototype uses the dataset's flat CSV files rather than a database. Logical structure:

Table	Key fields
projections	`uid`, `filename`, `projection` (Frontal / Lateral)
reports	`uid`, `Problems` (semicolon-separated tags), plus Findings/Impression text

A relational store (e.g., SQLite/PostgreSQL) is planned for the web phase to hold prediction history and the audit log.

2.6 Hardware Components

Training used a GPU runtime provided by Kaggle/Colab (code selects CUDA automatically, otherwise CPU). **[TO FILL: exact GPU model, RAM, and training time per epoch.]**

2.7 APIs and Integration Procedure

- **Current:** no external API; the model loads from a saved checkpoint (`best_densenet.pt`).
- **Planned:** FastAPI `POST /predict` accepting an uploaded image (and report text); the model is put into `eval()` mode and wrapped in `torch.no_grad()`, `unsqueeze(0)` adds the batch dimension, and `torch.sigmoid` converts logits to probabilities returned as JSON.

---

3. Prototype Development

3.1 Functional Modules Developed

1. Data ingestion and CSV merge by patient `uid`.
2. Automatic label discovery (top-14 frequent tags) and multi-hot encoding.
3. Patient-level 70/15/15 train/validation/test split.
4. Custom PyTorch `Dataset` (`ChestXrayDataset`) with on-demand image loading and `DataLoader` batching (batch size 32, 2 workers).
5. DenseNet121 transfer-learning classifier with class-weighted loss.
6. Training loop with per-epoch validation, per-label AUROC reporting and automatic best-model checkpointing.
7. Separate test-set evaluation that reloads the best checkpoint.

3.2 Current Prototype Status

The **image-only baseline is complete and evaluated on held-out test data**. The text encoder, fusion model, explainability, API and website are not yet implemented.

3.3 Implementation Milestones

Milestone	Status
Dataset study and EDA of tags	Done
Label construction and patient-level split	Done
Preprocessing and augmentation pipeline	Done
DenseNet121 training (10 epochs)	Done
Validation and test AUROC evaluation	Done
Text baseline, fusion, explainability	Planned
Website (front end + back end)	Planned

3.4 Evidence of Working Components

[TO FILL – insert screenshots of: (a) printed label list and fraction-positive-per-label check, (b) training log of 10 epochs, (c) per-label AUROC output, (d) test AUROC output, (e) saved checkpoint file.]

---

4. Dataset and Experimental Setup

4.1 Data Source

Indiana University Chest X-Rays (Open-I), Kaggle (`raddar/chest-xrays-indiana-university`): approximately 7,470 chest X-ray images (frontal and lateral views) paired with approximately 3,955 de-identified radiology reports. It is a standard academic benchmark for image–report research.

4.2 Dataset Size and Split

Item	Value
Total images	≈ 7,470 (both views used; frontal-only filter available but disabled)
Reports (studies)	≈ 3,955
Split ratio	70% train / 15% validation / 15% test, by patient `uid`
Random seed	42
Number of labels	14

[TO FILL: exact image counts printed by the code for train/val/test.]

4.3 Preprocessing Procedures

See Section 2.3. Additional steps: missing `Problems` filled as `normal`; images converted to 3-channel RGB to match the ImageNet-pretrained input; labels built once before training for speed.

4.4 Experimental Configuration

Parameter	Value
Backbone	DenseNet121 (ImageNet pretrained)
Input size	224×224×3
Output	14 logits (multi-label)
Loss	BCEWithLogitsLoss with capped `pos_weight` (max 20)
Optimizer	Adam
Learning rate	1e-4
Batch size	32
Epochs	10

Parameter	Value
Checkpoint rule	Save when validation mean AUROC improves
Layers frozen	None
DataLoader workers	2

4.5 Hardware / Software Environment

Python, PyTorch, torchvision, scikit-learn, pandas, NumPy, Pillow on Kaggle (training) and Google Colab (test evaluation). **[TO FILL: library versions and GPU model.]**

4.6 Evaluation Methodology

- Per-label ROC-AUC and macro-averaged mean AUROC on validation data after every epoch.
- Best checkpoint selected by validation mean AUROC; final unbiased estimate on the untouched test split.
- Accuracy is deliberately not used as a headline metric because labels are highly imbalanced.

---

5. Testing and Validation

5.1 Validation Procedures Built Into the Pipeline

#	Test / Check	Expected	Observed
T1	Merge on `uid` keeps image rows with their report tags	Every image row has a `Problems` value	Passed (missing values filled with `normal`)
T2	Label sanity: fraction positive per label	No label column is all zeros	Passed – labels have non-zero positives (training AUROC computed for all 14 labels)
T3	Batch shape check	Images `[32, 3, 224, 224]`, labels `[32, 14]`	Passed *[confirm from your printout]*

#	Test / Check	Expected	Observed
T4	Patient-level split	No `uid` appears in more than one split	Guaranteed by splitting unique `uid`s first
T5	Training convergence	Training loss decreases	Passed – 1.156 → 0.529 over 10 epochs
T6	Generalization	Validation loss decreases with training	**Partially failed** – validation loss rose after epoch 2 (overfitting, see 5.3)
T7	Checkpoint reload	Saved weights load correctly	Passed – "Successfully loaded model weights"
T8	Held-out test evaluation	Test mean AUROC close to validation	Passed – 0.7323 test vs. 0.738 best validation

5.2 Additional Test Scenarios (to be executed)

Scenario	Expected outcome	Status
Frontal-only vs. both views	Compare AUROC	Planned
Inference on a single uploaded image	Returns 14 probabilities in [0, 1]	Planned (needs API)
Invalid file upload (non-image)	Graceful error message	Planned
Grad-CAM sanity on textbook cases	Heatmap highlights lung fields, not borders/annotations	Planned

5.3 Identified Defects and Corrective Measures

Issue	Effect	Corrective measure
Image-level split risk (multiple images per patient)	Data leakage, inflated scores	Split by unique `uid`
Labels live in a different CSV from image filenames	Images without labels	Merge on `uid`

Issue	Effect	Corrective measure
Severe class imbalance	Model ignores rare findings	Class-weighted loss, capped at 20
Substring label matching could create false positives	Noisy labels	Exact tag matching
Labels rebuilt on every sample access	Slow training	Label matrix computed once in the constructor
Overfitting after epoch ~2–5	Validation loss climbs from 1.09 to 1.51	Best-checkpoint selection on validation AUROC; further fixes proposed in Section 8

---

6. Quantitative Results and Performance Analysis

Experimental conditions: DenseNet121 (ImageNet-pretrained, fully fine-tuned), 224×224 input, batch size 32, Adam (lr 1e-4), weighted BCE loss, 10 epochs, seed-42 patient-level 70/15/15 split, 14 labels, image-only input.

6.1 Training Progress

Epoch	Train loss	Val loss	Val mean AUROC
1	1.1563	1.1327	0.695
2	1.0365	1.0913	0.723
3	0.9710	1.0979	0.733
4	0.9085	1.1199	0.724
5	0.8533	1.1078	0.737
6	0.7839	1.1846	0.736
7	0.7250	1.1447	0.736
8	0.6693	1.2016	**0.738** (best)
9	0.5959	1.4032	0.729
10	0.5289	1.5077	0.724

Observation: training loss falls steadily, but validation loss is lowest at epoch 2 and then rises, while validation AUROC plateaus near 0.73–0.74. This is a classic overfitting pattern. The best validation AUROC (0.738) occurred at epoch 8, which is the checkpoint used for testing.

6.2 Per-Label Results

Label	Val AUROC (epoch 10)	**Test AUROC (best checkpoint)**
Cardiomegaly	0.853	**0.883**
Pleural Effusion	0.909	**0.874**
Diaphragm	0.686	0.810
Aorta	0.755	0.792
Pulmonary Atelectasis	0.765	0.784
Opacity	0.703	0.772
Lung	0.800	0.717

Label	Val AUROC (epoch 10)	**Test AUROC (best checkpoint)**
Markings	0.736	0.699
Thoracic Vertebrae	0.655	0.696
Calcinosis	0.656	0.687
Density	0.599	0.685
Cicatrix	0.715	0.682
Calcified Granuloma	0.652	0.595
Spine	0.657	0.576
Mean	–	**0.7323**

Note: the validation column is from the final epoch (10), not the best checkpoint, so the two columns are not directly comparable.

6.3 Analysis

- **Strongest findings:** Cardiomegaly (0.883) and Pleural Effusion (0.874) are large, visually distinct abnormalities that a CNN detects well; they are also clinically significant.
- **Weakest findings:** Spine (0.576) and Calcified Granuloma (0.595) are small, subtle or ambiguous structures, and several labels (Spine, Thoracic Vertebrae) are anatomical tags rather than true diagnoses, so their "presence" depends on how the radiologist chose to write the report.
- **Generalization:** Test mean AUROC (0.7323) is close to the best validation value (0.738), so the evaluation is consistent and there is no sign that model selection overfit the validation set.
- **Comparison to literature:** Published IU X-ray image-classification results typically fall in a similar range for DenseNet-style baselines; treat this only as a rough reference and verify before citing.

6.4 Metrics Not Yet Measured

Precision, recall, F1-score per finding, calibration (reliability diagram), inference latency, training time per epoch, throughput and GPU memory utilization have **not yet been recorded**. *[TO FILL once measured: e.g., latency per image in ms, epoch time in seconds.]*

---

7. Integration and Verification

7.1 Integration Status

Integration	Status	Evidence
Data CSVs → Dataset → DataLoader	Integrated and verified	Batch shapes printed; training runs end-to-end
DataLoader → DenseNet121 → Loss/Optimizer	Integrated and verified	Loss decreases across 10 epochs
Training → Validation → Checkpoint	Integrated and verified	`best_densenet.pt` saved on AUROC improvement
Checkpoint → Test evaluation	Integrated and verified	Weights reloaded; test AUROC computed
Text encoder ↔ Vision encoder (fusion)	Not started	–
Model ↔ FastAPI ↔ Dashboard	Not started	–

7.2 Interface Compatibility

Tensor shapes between modules are consistent: loader output `[B, 3, 224, 224]` matches the model input; the model emits `[B, 14]` logits matching the `[B, 14]` label tensor. For future fusion, the 1024-d vision vector and 768-d text vector are compatible with a concatenated 1792-d fusion input.

7.3 Functional and System-Level Verification

The vision pipeline has been verified at module level and as a complete train-validate-test chain. System-level end-to-end testing (upload → prediction → explanation) will be performed after the API and website are built.

---

8. Technical Challenges and Limitations

8.1 Challenges Faced

1. **New domain.** The team was previously familiar with text/tabular data; working with images was new, and the CNN model, its tensor structure, transfer learning, and the PyTorch training loop had to be learned from scratch.
2. **Label quality.** The dataset provides report tags, not clean diagnostic labels, so the label set mixes diagnoses (Cardiomegaly, Pleural Effusion) with anatomical terms (Spine, Aorta).
3. **Class imbalance.** Most findings are rare.
4. **Small dataset.** About 7,470 images are small for a DenseNet121 with ~7M parameters.

8.2 Limitations and Unresolved Issues

- **Overfitting:** validation loss increases from epoch 3 onward.
- **Image-only model:** the radiology report text, a major information source, is not yet used.
- **Both views treated independently:** frontal and lateral images of one study are scored separately; there is no study-level aggregation.
- **Mixed-quality labels:** weak labels cap achievable AUROC for several findings.
- **Single-source data:** one hospital network, demographic skew, and no clinical validation.
- **Incomplete metrics:** no F1/precision/recall, calibration, or latency measurements yet.
- **Not a medical device:** this is a research/decision-support prototype only.

8.3 Proposed Corrective Strategies

Problem	Proposed fix
Overfitting	Early stopping (best epoch 2–8), stronger augmentation, freeze early layers (`denseblock4` + head only), weight decay, learning-rate scheduler, or lower epoch count
Weak labels	Use the CheXpert labeler on Findings/Impression text for cleaner positive/negative/uncertain labels

Problem	Proposed fix
Small data	Enlarge the dataset (COVID-19 Radiography, Chest X-Ray Pneumonia, or MIMIC-CXR as stretch)
Rare findings	Focal loss or oversampling
Two views per study	Average or max-pool predictions across a study's images
Trust and calibration	Temperature scaling, reliability diagram, Grad-CAM sanity checks

---

9. Milestone Achievement and Future Work

9.1 Completed

- Dataset acquisition, exploration and tag analysis
- Label construction (14 findings) and patient-level 70/15/15 split
- Image preprocessing and augmentation pipeline
- DenseNet121 transfer-learning baseline: trained, validated, tested (**test mean AUROC 0.7323**)
- Per-label evaluation and best-checkpoint saving

9.2 Partially Implemented

- Evaluation: AUROC done; precision/recall/F1 and calibration pending
- Class-imbalance handling: weighted loss done; focal loss/oversampling not yet tried

9.3 Ongoing

- Learning and tuning of CNN training behaviour (reducing overfitting)
- Compiling results, screenshots and documentation

9.4 Planned (Future Work)

Phase	Task	Description
A	Overfitting control and metric completion	Early stopping, partial freezing, F1/precision/recall, calibration
B	Text-only baseline	Bio_ClinicalBERT on report text with the same labels
C	Multimodal fusion	Concatenation of 1024-d and 768-d vectors plus MLP head; compare against both unimodal baselines
D	Explainability	Grad-CAM on `denseblock4`; attention visualization for text
E	Dataset expansion	Larger/additional datasets to improve generalization
F	**Back end**	FastAPI service exposing `/predict`, model loading, input validation, audit log
G	**Front end / website**	Clinician dashboard for uploading X-ray and report and viewing predictions, confidence and heatmap
H	Testing and deployment	System-level tests, Docker packaging, live demo, final documentation and demo video

9.5 Remaining Activities

Development: text encoder, fusion model, API and website. Testing: end-to-end and invalid-input tests. Optimization: regularization and hyper-parameter tuning, inference latency. Validation: calibration and explainability checks, plus a documented limitations and ethics statement.

---

10. Conclusion

The project has completed its first major technical milestone: a working, patient-level-validated image-only DenseNet121 classifier for 14 chest X-ray findings, reaching a **test mean AUROC of 0.7323**, with strong performance on clinically important findings such as

Cardiomegaly (0.883) and Pleural Effusion (0.874). The remaining work extends this baseline into the full multimodal CDSS with text fusion, explainability, a back end and a website.

This system is a decision-support research prototype and has not undergone clinical validation; it must not be used for real diagnosis.